S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 10

Aim: Apply prompt engineering to construct a prompt for learning the concept of Adversarial Search.

Name of Student:

Roll No.:

Semester/Year: V/III

Academic Session:

Date of Performance:

Date of Submission:

AIM: To design and refine an effective prompt using prompt engineering techniques for learning the concept of Adversarial Search from Artificial Intelligence.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of a *prompt* (input given to an AI model).
- To be able to understand the concept of *prompt engineering* (designing inputs to get desired outputs)
- To be able to work with basic idea and models like ChatGPT.
- To be able to distinguish between a vague prompt and a well-structured one.

THEORY:

i. Fundamentals of Prompt Engineering

Prompt engineering is the process of designing and structuring inputs (prompts) to get accurate, relevant, and well-formatted outputs from AI systems such as ChatGPT. Prompt engineering is how you ask a question to AI so that it gives the best possible answer.

ii. Characteristics of a Good Prompt

A good prompt is based on the following core principles:

- **Clarity** → Avoid ambiguity
- **Specificity** → Define exact task
- **Context** → Provide background
- **Constraints** → Limit output (length, format, style)
- **Intent** → Clearly state what is required

Example:

- Weak: *"Explain AI"*
- Strong: *"Explain AI in 5 bullet points with real-world examples for beginners"*

iii. Prompt Patterns / Techniques

The common patterns of the prompt along with their usage are as follows:

a) Role-Based Prompting

"You are an AI professor…"

b) Step-by-Step Prompting

"Explain step-by-step…"

c) Few-shot Prompting

Provide examples in the prompt

d) Instruction + Constraints

"Explain in 100 words using simple language"

e) Chain-of-Thought (Guided Reasoning)

"Show reasoning before final answer"

iv. Task-Oriented Prompting

Based on the tasks we can have different ways to provide the prompt. Following are the examples to design prompts for different tasks:

- **Learning/Explanation**
- **Coding**

- **Summarization**
- **Question generation**
- **Problem solving (AI, Cloud, etc.)**

v. Iterative Refinement (Very Important)

The first prompt may not be the best one. So, we must learn how the prompt can be refined to get most appropriate outcome:

- First prompt ≠ best prompt
- Improve by:
  - Adding constraints
  - Asking follow-up questions
  - Correcting output

Concept:

Prompt → Output → Refine → Better Output

vi. Evaluation of AI Output

It is very important to evaluate the output generated by AI. Critical thinking should be done to find the answer to the following:

- Is the answer **correct**?
- Is it **complete**?
- Is it **biased or vague**?
- Does it follow instructions?

vii. Ethical and Responsible Use

- Avoid Misuse and Plagiarism
  - Do not copy AI-generated content blindly for assignments or research
  - Always ensure originality and proper attribution when required
- Verify Information
  - AI tools like ChatGPT may generate incorrect or outdated information
  - Cross-check important facts before using them in academic or professional work
- Use AI as a Support Tool
  - AI should assist learning, not replace critical thinking
  - Students should understand concepts rather than depend entirely on AI-generated answers

viii. Basic Applications in Academics

- Generate notes
- Create MCQs and scenario-based questions
- Debug code
- Explain complex topics simply

ix. Perform the following tasks:

- Convert a bad prompt → good prompt
- Generate:
  - Notes
  - Quiz questions
  - Code snippets
- Design a prompt for:

- Adversarial Search explanation

EXPECTED OUTPUT:

- Definition
- Game tree explanation
- Minimax working
- Alpha-Beta optimization
- Example
- Complexity analysis

INPUT & OUTPUT:
Design the prompts and provide the analysis using the following criteria:

Attempt	Prompt Quality	Output Quality	Improvements
1	Basic	Vague	Add structure
2	Improved	Better	Add constraints
3	Refined	High quality	Final

1. Explain adversarial search and implement a Nim game using it. Include: - Definition of adversarial search - Game tree explanation - Minimax algorithm - Alpha-beta pruning - One example - Complexity analysis Also provide Python code for the Nim game using minimax.

1. Definition of Adversarial Search

Adversarial search is used in AI for **competitive environments** where one player tries to maximize their win while the opponent tries to minimize it (e.g., games like Nim, chess).

2. Game Tree (Nim)

A **game tree** represents all possible moves from a current state.

- Each node = a game state (pile sizes)
- Edges = possible moves
- Leaves = end states (win/loss)

Example (small):

```
(3 stones)
├── (2)
│   ├── (1)
│   │   └── (0) ✔
```

└── (1)
  └── (0) ✔

3. Minimax Algorithm

- Two players: **Max (AI)** and **Min (Opponent)**
- Max tries to maximize score (+1 win)
- Min tries to minimize score (-1 loss)

Steps:

1. Generate game tree
2. Assign values to terminal states
3. Propagate values upward
4. Choose best move

4. Alpha-Beta Pruning

- Optimization of Minimax
- Avoids exploring unnecessary branches
- Uses:
  - **Alpha** = best value for Max
  - **Beta** = best value for Min
- Prunes when **alpha ≥ beta**

 Reduces computation significantly

5. Example (3 piles: [1,2,3])

- Players remove stones from any pile
- AI evaluates all moves using Minimax
- Chooses move that forces opponent into losing state

6. Complexity Analysis

- Minimax Time: **O(b^d)**
- Space: **O(d)**
- Alpha-Beta: **O(b^(d/2))** (best case)

□ Python Implementation (Nim with Minimax)

```
import math

# Check if game over
def is_terminal(piles):
    return all(p == 0 for p in piles)

# Evaluate state
```

```
def evaluate(piles, is_max):
    if is_terminal(piles):
        return -1 if is_max else 1
    return 0

# Generate possible moves
def get_moves(piles):
    moves = []
    for i in range(len(piles)):
        for j in range(1, piles[i] + 1):
            new_piles = piles.copy()
            new_piles[i] -= j
            moves.append(new_piles)
    return moves

# Minimax function
def minimax(piles, depth, is_max):
    if is_terminal(piles):
        return evaluate(piles, is_max)

    if is_max:
        best = -math.inf
        for move in get_moves(piles):
            val = minimax(move, depth + 1, False)
            best = max(best, val)
        return best
    else:
        best = math.inf
        for move in get_moves(piles):
            val = minimax(move, depth + 1, True)
            best = min(best, val)
        return best

# Best move for AI
def best_move(piles):
    best_val = -math.inf
    best_state = None
    for move in get_moves(piles):
        val = minimax(move, 0, False)
        if val > best_val:
            best_val = val
            best_state = move
    return best_state

# Game loop
def play_nim():
    piles = [1, 2, 3]
    print("Initial piles:", piles)

    while not is_terminal(piles):
        # Human move
        i = int(input("Choose pile index: "))
        remove = int(input("Stones to remove: "))
        piles[i] -= remove
        print("After your move:", piles)

        if is_terminal(piles):
            print("You win!")
            break
```

```
    # AI move
    piles = best_move(piles)
    print("AI move:", piles)

    if is_terminal(piles):
        print("AI wins!")
        break

play_nim()
```

2 Improved prompt

2. You are an AI expert. Explain adversarial search using the Nim game. Include: definition, game tree explanation, Minimax working (step-by-step), alpha-beta pruning, a small example (3 piles), and complexity analysis. Then provide a simple, well-commented Python implementation of Nim using Minimax (with optional alpha-beta pruning) supporting human vs AI gameplay.

1. Definition of Adversarial Search

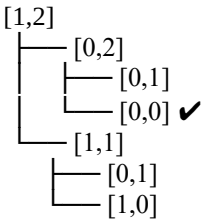
Adversarial search is used in AI for **two-player competitive games**, where one player (Max) tries to **maximize** the outcome and the opponent (Min) tries to **minimize** it.

2. Game Tree (Nim)

A **game tree** shows all possible states of the game:

- Node = current piles
- Edge = removing stones
- Leaf = game over

Example:



3. Minimax Algorithm (Step-by-Step)

1. Generate all possible moves
2. Go to terminal states (win/loss)
3. Assign values: +1 (win), -1 (loss)
4. Backtrack values:

- Max chooses highest
- Min chooses lowest

5. Select best move for AI

4. Alpha-Beta Pruning

- Improves Minimax by **cutting unnecessary branches**
- Uses:
  - Alpha = best for Max
  - Beta = best for Min
- Stop exploring when **alpha ≥ beta**
  □ Faster decision-making

5. Example (Piles: [1,2,3])

- AI explores all moves
- Evaluates outcomes using Minimax
- Picks move that forces opponent into losing state

6. Complexity Analysis

- Minimax Time: **O(b^d)**
- Space: **O(d)**
- Alpha-Beta Time: **O(b^(d/2))** (best case)

□ Python Implementation (Nim with Minimax + Alpha-Beta)

```
import math

# Check terminal state
def is_terminal(piles):
    return all(p == 0 for p in piles)

# Generate moves
def get_moves(piles):
    moves = []
    for i in range(len(piles)):
        for j in range(1, piles[i] + 1):
            new_piles = piles.copy()
            new_piles[i] -= j
            moves.append(new_piles)
    return moves

# Minimax with Alpha-Beta
def minimax(piles, alpha, beta, is_max):
    if is_terminal(piles):
        return -1 if is_max else 1

    if is_max:
        best = -math.inf
        for move in get_moves(piles):
```

```
            val = minimax(move, alpha, beta, False)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break  # Prune
        return best
    else:
        best = math.inf
        for move in get_moves(piles):
            val = minimax(move, alpha, beta, True)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break  # Prune
        return best

# Find best move
def best_move(piles):
    best_val = -math.inf
    best_state = None
    for move in get_moves(piles):
        val = minimax(move, -math.inf, math.inf, False)
        if val > best_val:
            best_val = val
            best_state = move
    return best_state

# Game loop
def play_nim():
    piles = [1, 2, 3]
    print("Initial piles:", piles)

    while not is_terminal(piles):
        # Human move
        i = int(input("Choose pile index: "))
        remove = int(input("Stones to remove: "))
        if piles[i] >= remove:
            piles[i] -= remove
        else:
            print("Invalid move!")
            continue

        print("After your move:", piles)
        if is_terminal(piles):
            print("You win!")
            break

        # AI move
        piles = best_move(piles)
        print("AI move:", piles)

        if is_terminal(piles):
            print("AI wins!")
            break

play_nim()
```

3. refined prompt

3. Act as an AI professor and programmer. Teach adversarial search using the Nim game with clearly labeled sections: definition, game tree (with small diagram), Minimax step-by-step, alpha-beta pruning (with comparison), example (3 piles), and complexity analysis. Then implement a clean, modular, well-commented Python Nim game using Minimax with alpha-beta pruning and human vs AI mode.

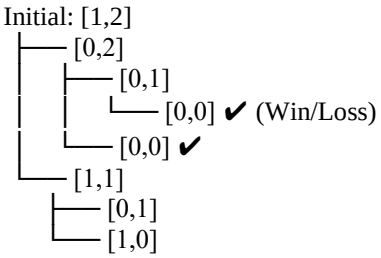
□ 1. Definition

Adversarial search is an AI technique used in **competitive environments** where two players make decisions:

- **Max (AI)** → tries to maximize score
- **Min (Opponent)** → tries to minimize score

□ 2. Game Tree (Nim Representation)

A **game tree** shows all possible states of the Nim game.



- Nodes = pile states
- Leaves = terminal states (game over)

□ 3. Minimax (Step-by-Step)

1. Generate all possible moves
2. Go to terminal states
3. Assign values:
   - +1 → AI wins
   - -1 → AI loses
4. Backtrack values:
   - Max picks highest
   - Min picks lowest
5. Choose optimal move

✂ 4. Alpha-Beta Pruning (Optimization)

- Reduces unnecessary exploration
- Uses:
  - **Alpha (α)** = best for Max
  - **Beta (β)** = best for Min

 If **α ≥ β → prune branch**

Comparison:

- Minimax explores all nodes
- Alpha-Beta skips useless branches → faster

5. Example (Piles: [1,2,3])

- AI evaluates all moves
- Predicts opponent responses
- Chooses move that guarantees a **winning path**

6. Complexity Analysis

- Minimax Time: **O(b^d)**
- Space: **O(d)**
- Alpha-Beta Time: **O(b^(d/2))** (best case)

7. Python Implmentation (Modular & Clean)

```
import math

# ------------------------------
# Utility Functions
# ------------------------------

def is_terminal(piles):
    """Check if all piles are empty"""
    return all(p == 0 for p in piles)

def get_moves(piles):
    """Generate all possible moves"""
    moves = []
    for i in range(len(piles)):
        for j in range(1, piles[i] + 1):
            new_piles = piles.copy()
            new_piles[i] -= j
            moves.append(new_piles)
    return moves

# ------------------------------
# Minimax with Alpha-Beta
# ------------------------------
```

```
def minimax(piles, alpha, beta, is_max):
    """Minimax algorithm with alpha-beta pruning"""

    if is_terminal(piles):
        return -1 if is_max else 1

    if is_max:
        max_eval = -math.inf
        for move in get_moves(piles):
            eval = minimax(move, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break  # Pruning
        return max_eval
    else:
        min_eval = math.inf
        for move in get_moves(piles):
            eval = minimax(move, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break  # Pruning
        return min_eval

# ----------------------------
# AI Move Selection
# ----------------------------

def best_move(piles):
    """Find the best move for AI"""
    best_val = -math.inf
    best_state = None

    for move in get_moves(piles):
        move_val = minimax(move, -math.inf, math.inf, False)
        if move_val > best_val:
            best_val = move_val
            best_state = move

    return best_state

# ----------------------------
# Game Loop (Human vs AI)
# ----------------------------

def play_nim():
    piles = [1, 2, 3]
    print("Initial piles:", piles)

    while not is_terminal(piles):

        # Human Turn
        print("\nYour turn")
        print("Piles:", piles)
        i = int(input("Choose pile index (0,1,2): "))
        stones = int(input("Stones to remove: "))

        if i < 0 or i >= len(piles) or stones > piles[i] or stones <= 0:
```

```
            print("Invalid move! Try again.")
            continue

        piles[i] -= stones

        if is_terminal(piles):
            print("You win!")
            break

        # AI Turn
        print("\nAI is thinking...")
        piles = best_move(piles)
        print("AI move:", piles)

        if is_terminal(piles):
            print("AI wins!")
            break

# Run the game
play_nim()
```

Analysis :

1. Basic Prompt:

- Produces general explanation with minimal depth
- May miss step-by-step clarity
- Code likely simple but less structured
- Limited examples and weak optimization explanation
- Good for quick, basic understanding

2. Improved Prompt:

- More structured and detailed explanation
- Includes step-by-step Minimax and clear example
- Better explanation of alpha-beta pruning
- Code is cleaner, readable, and usable
- Balanced for learning + implementation

3. Refined Prompt:

- Highly organized, section-wise detailed output
- Deep conceptual clarity with diagrams & comparisons
- Strong Minimax + alpha-beta explanation
- Clean, modular, professional-level code
- Best for exams, projects, and thorough understanding

CONCLUSION:

DISCUSSION QUESTIONS:

1. What is prompt engineering?

2. Why is prompt refinement important?

3. What is Alpha-Beta pruning?

4. Difference between weak and strong prompts?

REFERENCES:

- *https://learnpromptengineering.dev*
- *https://www.linkedin.com/pulse/40-prompt-engineering-resources-upskilling-spencer-taylor-xje0e/*
- *https://justprompting.in/*